

Resenha - Design by Contract

Bertrand Meyer

Bertrand Meyer argumenta que a confiabilidade do software, composta pela correção e pela robustez, é um dos desafios mais prementes da engenharia de software moderna. Meyer sustenta que, embora a orientação a objetos ofereça estruturas flexíveis, ela frequentemente silencia sobre como garantir que esses componentes reutilizáveis funcionem conforme o esperado. A metáfora do contrato serve como um fundamento metodológico para documentar decisões de design, tratando a construção de software como uma sucessão de acordos precisos entre elementos clientes e fornecedores.

Características da Linguagem

O "Design by Contract" (DbC) é estruturado sobre a premissa de que uma relação entre dois módulos deve ser regida por um documento de contrato que especifica obrigações e benefícios mútuos:

Precondições: Requisitos que o cliente deve satisfazer para que a chamada seja considerada correta; se violadas, o erro é atribuído ao cliente;

Invariantes de Classe: Restrições de integridade que transcendem rotinas individuais, devendo ser preservadas por todas as operações exportadas da classe;

Semântica de Exceções: Ao contrário de abordagens ad hoc, no DbC uma exceção só ocorre quando um contrato não pode ser cumprido, resultando em "pânico organizado" (falha) ou retomada (resumption);.

Principais Aplicações

Meyer destaca que a aplicação consistente de asserções transforma o desenvolvimento, impactando o ciclo de vida do software:

Documentação Abstrata: As asserções permitem extrair interfaces claras (via ferramentas como o comando short) que explicam o "o quê" sem revelar detalhes de implementação;

Depuração e Validação: Durante o desenvolvimento, o monitoramento de asserções serve como um mecanismo de controle de qualidade superior ao teste "cego", detectando inconsistências lógicas em tempo de execução;

Habilitação da Herança Segura: Define regras para redefinição de rotinas (subcontratação), onde descendentes podem apenas enfraquecer precondições e fortalecer pós-condições, garantindo o polimorfismo seguro;

Substituição da Programação Defensiva: Elimina códigos de verificação redundantes e desnecessários, distribuindo responsabilidades de forma clara e simplificando a arquitetura global do sistema;

O capítulo demonstra que a construção de software baseada em asserções fornece o rigor necessário para transformar o design orientado a objetos em sistemas comprovadamente confiáveis. Através do uso de precondições, pós-condições e invariantes de classe, a metodologia estabelece uma divisão de responsabilidades clara que elimina a complexidade da programação defensiva e simplifica o tratamento de exceções. O Design by Contract serve como a "ponte de integridade" que permite que o design de alto nível seja verificado e validado, garantindo que a herança e o polimorfismo sejam aplicados de forma segura e que o sistema mantenha estados consistentes do modelo até a implementação final.

Sistema de Gestão Financeira

No contexto de um sistema de gestão para projetos sociais, o uso do DbC garante que as transações financeiras sigam regras de negócio rigorosas. Por meio de uma invariante, o sistema impede estados inválidos, como o saldo total de uma conta tornar-se inconsistente com a soma de seus lançamentos. Por meio de precondições, rotinas de pagamento são protegidas contra entradas inválidas (como valores negativos), sem a necessidade de fluxos de erro complexos dentro da lógica principal. Por fim, o uso de pós-condições assegura que, após o registro de uma despesa, o sistema confirme o novo estado estável dos dados, resultando em um modelo autoexplicativo, robusto e livre de bugs silenciosos.